Previous Term Paper

Sunday, 30 March, 2025 12:30

Consider the following functions:

$h1(n) = 10n^2 \log n + n^2$

$h2(n) = \log(\log n)$

$h3(n) = 100n$

$h4(n) = 2^n + n \log n$

$h5(n) = n^2 + 1000$

Arrange the above functions in increasing order of asymptotic complexity.

Options :

[redacted] $h2(n), h3(n), h1(n), h4(n), h5(n)$

[redacted] $h2(n), h3(n), h5(n), h1(n), h4(n)$

[redacted] $h2(n), h4(n), h5(n), h3(n), h1(n)$

[redacted] $h2(n), h3(n), h1(n), h5(n), h4(n)$

h1(n) $O(n^2 \log n)$ 4

h2(n) $O(\log(\log n))$ 1

h3(n) $O(n)$ 2

h4(n) $O(2^n)$ 5

h5(n) $O(n^2)$ 3

A list with 2^k elements needs to be processed using either of two given algorithms. Algorithm **A** takes $5n \log_2 n$ time and algorithm **B** takes $0.02n^2$ time to process a list of n elements. What is the smallest value of k for which algorithm **A** would be preferred?

Options :

6406533043418. ✖ 10

6406533043419. ✖ 11

6406533043420. ✔ 12

6406533043421. ✖ 13

A $O(n \log n)$ B $O(n^2)$

$\frac{2^{12}}{12} = 341.33$

$\frac{2^k}{k} >= 250$

$k = 12$

$5 \leq k \leq 0.02 \, 2^{\frac{1}{2}k}$

$\frac{5}{0.02} <= \frac{2^k}{k}$

$\frac{2^k}{k} >= 250$

Consider the following recursive function to find the minimum element in list `L` of size `n` where `lower` and `upper` represents the first index and last index of list `L` respectively.

```
1  def find_min(L, lower, upper):
2      if upper-lower == 0:
3          return L[lower]
4      return min(L[lower],find_min(L,lower+1,upper))
```

lower == upper

What is the Recurrence relation of the given function?

Options :

- [redacted] $T(n) = T(n/2) + 1 \, , \, T(1) = 1$
- [redacted] $T(n) = T(n-1) + 1 \, , \, T(1) = 1$
- [redacted] $T(n) = T(n-1) + n \, , \, T(1) = 1$
- [redacted] $T(n) = T(n/2) + n \, , \, T(1) = 1$

1 2 4 3 5

min(1, min(2, min(4, min(3, 5))))

$\Rightarrow 1$ $O(1)$

Given the following sorted list :

```
[16, 53, 59, 81, 94, 99, 121, 150, 162, 170, 187]
 0   1   2   3   4   5   6    7    8    9   10
```

If we use binary search algorithm to search for element 53 in the given list, then the number of list elements for comparison to 53 (including comparison with 53 in list) in this process is 4

Note: Assume here that binary search will compute the midpoint by using $(First\ index + Last\ index)//2$

1) low = 0, high = 10, mid = 5

2) low = 0, high = 4, mid = 2

3) low = 0, high = 1, mid = 0

4) low = 1, high = 1, mid = 1

The list of tuples `L = [('a',2),('b',2),('c',4),('d',3),('e',2),('f',4),('g',1),('h',1)]` need to be sorted. The following `Insertionsort` function is executed on the list `L` which sorts the list based on the 2nd element of tuples.

```
1  def Insertionsort(L):
2      n = len(L)
3      if n < 1:
4          return(L)
5      for i in range(n):
6          j = i
7          while(j > 0 and L[j][1] < L[j-1][1]):
8              (L[j],L[j-1]) = (L[j-1],L[j])
9              j = j-1
10     return(L)
```

Options :

- [redacted] [('g', 1), ('h', 1), ('e', 2), ('a', 2), ('b', 2), ('d', 3), ('f', 4), ('c', 4)]
- [redacted] [('g', 1), ('h', 1), ('a', 2), ('b', 2), ('e', 2), ('d', 3), ('c', 4), ('f', 4)]
- [redacted] [('g', 1), ('h', 1), ('e', 2), ('b', 2), ('a', 2), ('d', 3), ('c', 4), ('f', 4)]
- [redacted] [('h', 1), ('g', 1), ('e', 2), ('b', 2), ('a', 2), ('d', 3), ('f', 4), ('c', 4)]

Which of the following list is returned by the function?

insertion sort is stable!

What is recurrence and time complexity for the **worst case** of **Quick Sort** ? Consider that algorithm select last element as pivot element.

Options :

- [redacted] Recurrence is $T(n) = T(n/2) + O(n)$ and time complexity is $O(n^2)$
- [redacted] Recurrence is $T(n) = T(n-1) + O(n)$ and time complexity is $O(n^2)$
- [redacted] Recurrence is $T(n) = 2T(n/2) + O(n)$ and time complexity is $O(n \log n)$
- [redacted] Recurrence is $T(n) = 2T(n/2) + O(1)$ and time complexity is $O(n \log n)$

$$T(n) = T(n-1) + O(n)$$

$$O(n^2)$$

There is a stack `S` and a queue `Q`. Suppose the elements A, B, C, D, E, F, G, H and I are enqueued into `Q` in the reverse order i.e., starting from I. The following operations are performed on the stack and the queue.

```
1  S.push(Q.dequeue())
2  S.push(Q.dequeue())
3  Q.enqueue(S.pop())
4  Q.enqueue(S.pop())
5  S.push(Q.dequeue())
6  S.push(Q.dequeue())
7  S.push(Q.dequeue())
8  Q.enqueue(S.pop())
9  S.push(Q.dequeue())
```

Options :

6406533043431. ✖ I

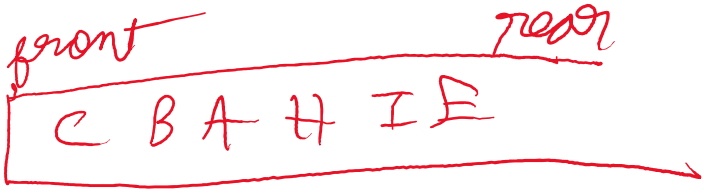
6406533043432. ✖ F

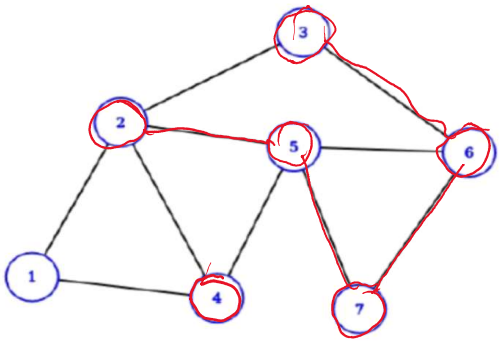
6406533043433. ✔ E

6406533043434. ✖ H

6406533043435. ✖ G

Which of the following element is the rear of queue after execution of the above operations?





Which of the following vertex sequence is the correct **BFS traversal** on the graph started from node **7**? Assume that when a node has multiple neighbours, BFS would visit the numerically smaller valued node first.

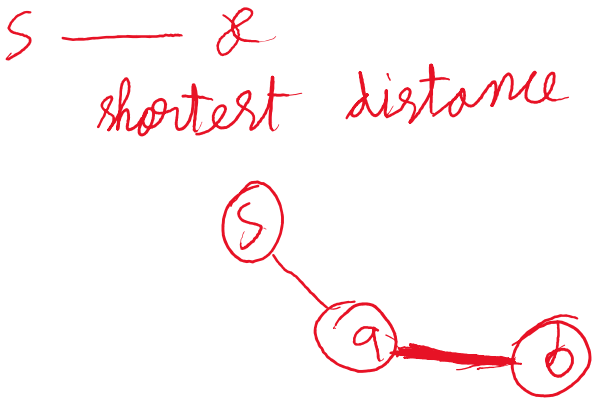
Options :

- [redacted] $7, 5, 6, 2, 3, 4, 1$
- [redacted] $7, 5, 6, 2, 4, 1, 3$
- [redacted] $7, 5, 6, 3, 4, 2, 1$
- [redacted] $7, 5, 6, 2, 4, 3, 1$

Let T_B and T_D be the BFS tree and DFS tree respectively generated when BFS and DFS are applied on the node s in undirected and unweighted graph G. Let $d(x)$ be the shortest distance of node x from the node s in G. Which among the following statements is/are correct ?

Options :

- [redacted] For every neighbor v of s in graph G, the edge (s, v) must exist in T_D.
- [redacted] If (u, v) is an edge of G that is not in T_B then $|(d(u) - d(v)| \leq 1$.
- [redacted] If there is no path from s to u in T_D, then u is in a different component from s.
- [redacted] The number of vertices in T_B and T_D is equal.



A baker is preparing an elaborate cake M. The recipe includes preparing several other components, each of which has its dependencies. The order in which these components must be prepared is given below:

1. Component X is used to make components P and Q.
2. Component Y is added to prepare components R and S.
3. Component T is prepared by mixing components Q and R.
4. Component Y is made by blending X.
5. Component U is made by mixing P and T.
6. Component V is made by adding sugar to component S.
7. The cake M is assembled by layering components U and V together.

Options :

6406533043444. ✖ Q

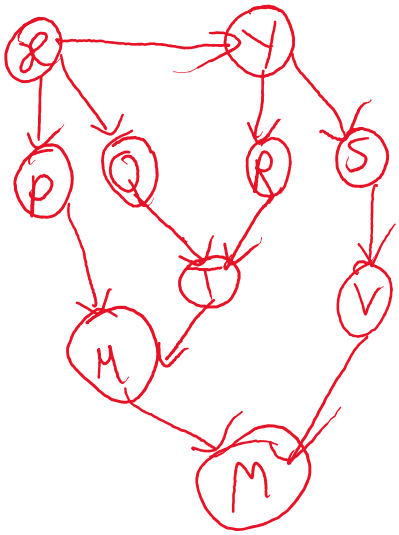
6406533043445. ✖ T

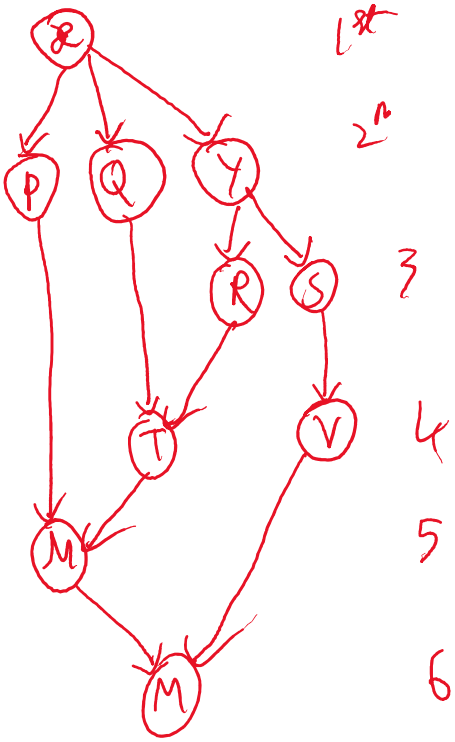
6406533043446. ✔ R

6406533043447. ✖ U

6406533043448. ✔ S

The baker has enough assistants to prepare multiple components simultaneously, allowing the cake M to be completed in the minimum number of steps, considering all dependencies. Each step represents a time unit during which one or more components can be prepared in parallel. The component(s) prepared in the 3rd step is/are ___.





Which of the following statements is **true** about Dijkstra's algorithm to find the shortest path?

I. The shortest path returned by Dijkstra's algorithm always passes through the least number of vertices.

II. To decide which node to visit next, Dijkstra's algorithm selects the node with maximum known distance.

Options :

6406533043449. ✖ Only I is correct

6406533043450. ✖ Only II is Correct

6406533043451. ✖ Both I and II are correct

6406533043452. ✔ Both I and II are incorrect

Which of the following is/are always **true** about the **Bellman-Ford** algorithm?

I. It can not detect negative weight cycles in graph.

II. It works correctly if the graph has negative edge weights but does not have negative weight cycles.

III. It finds the shortest paths from a single source vertex to all other vertices in the graph.

Options :

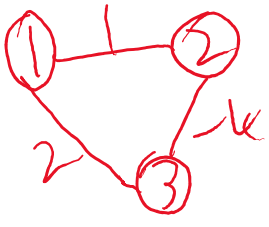
6406533043453. ✖ Only statement I and II are correct

6406533043454. ✖ Only statement I and III are correct

6406533043455. ✔ Only statement II and III are correct

6406533043456. ✖ All statements are correct

6406533043457. ✖ All statements are incorrect



n-1 passes - converge

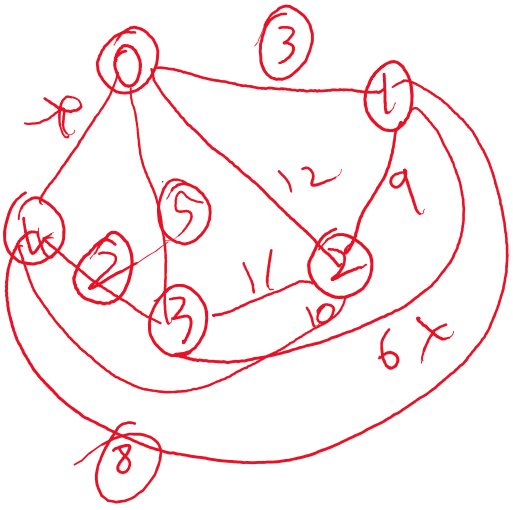
nth pass if something changes then -ve cycle

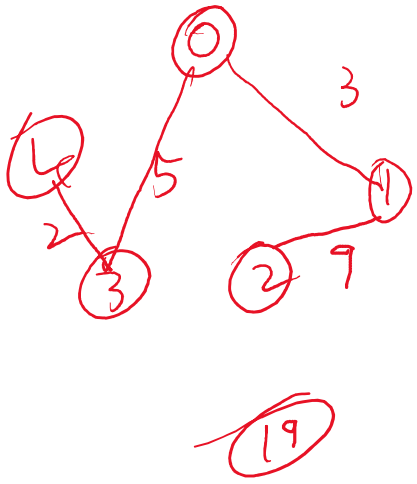
Consider a complete undirected graph with vertex set {0, 1, 2, 3, 4}. Every entry `w[i][j]` where `i` $\neq$ `j` in the matrix `w` below is the weight of the edge from vertex `i` to vertex `j`.

$$W = \begin{bmatrix} 0 & 3 & 12 & 5 & 7 \\ 3 & 0 & 9 & 6 & 8 \\ 12 & 9 & 0 & 11 & 10 \\ 5 & 6 & 11 & 0 & 2 \\ 7 & 8 & 10 & 2 & 0 \end{bmatrix}$$

adjacency matrix

What is the weight of the minimum spanning tree for the given graph? 19





Post-order traversal of a given binary search tree T produces the following sequence of keys:

`2, 6, 11, 8, 5, 14, 12, 20, 30, 25, 15`

Left child of element `14` is__.

Options :

6406533043460. ✖ 12

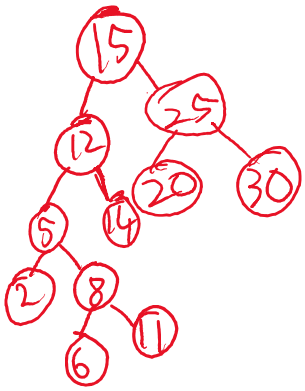
6406533043461. ✖ 11

6406533043462. ✖ 8

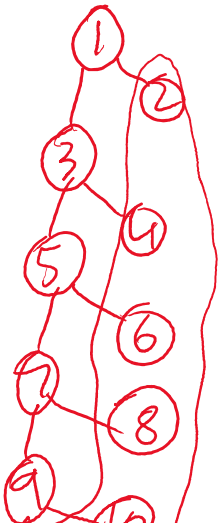
6406533043463. ✔ 14 is a ~~leaf node~~

Post – order

LRD

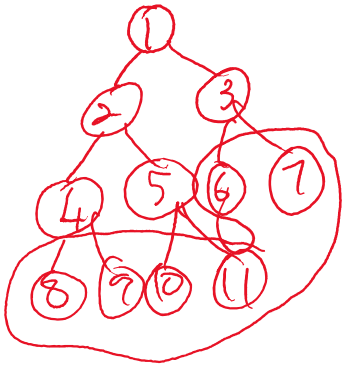


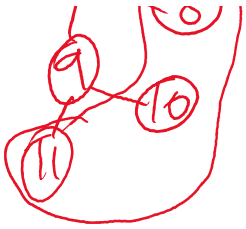
The number of leaf nodes in a rooted tree of `11` nodes, with each node having 0 or 2 children is 6.



2n-1

6





n leafs

Consider a binary **min-heap** implemented using list. Which of the following lists represents a binary min-heap?

Options :

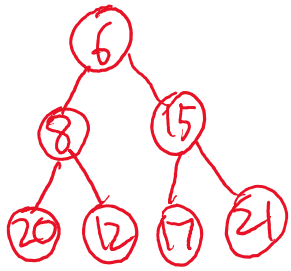
6406533043464. ✔ [6, 8, 15, 20, 12, 17, 21]

6406533043465. ✖ [6, 8, 17, 20, 12, 15, 21]

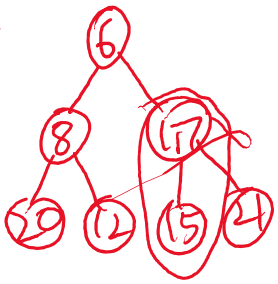
6406533043466. ✖ [6, 8, 15, 20, 17, 12, 21]

6406533043467. ✔ [6, 8, 17, 12, 15, 20, 21]

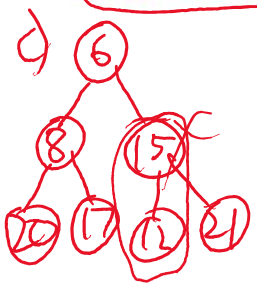
A)



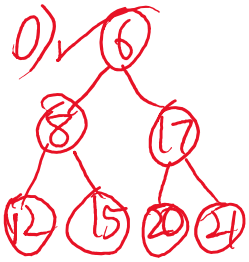
B)

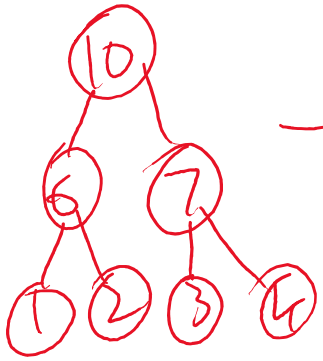


C)



D)



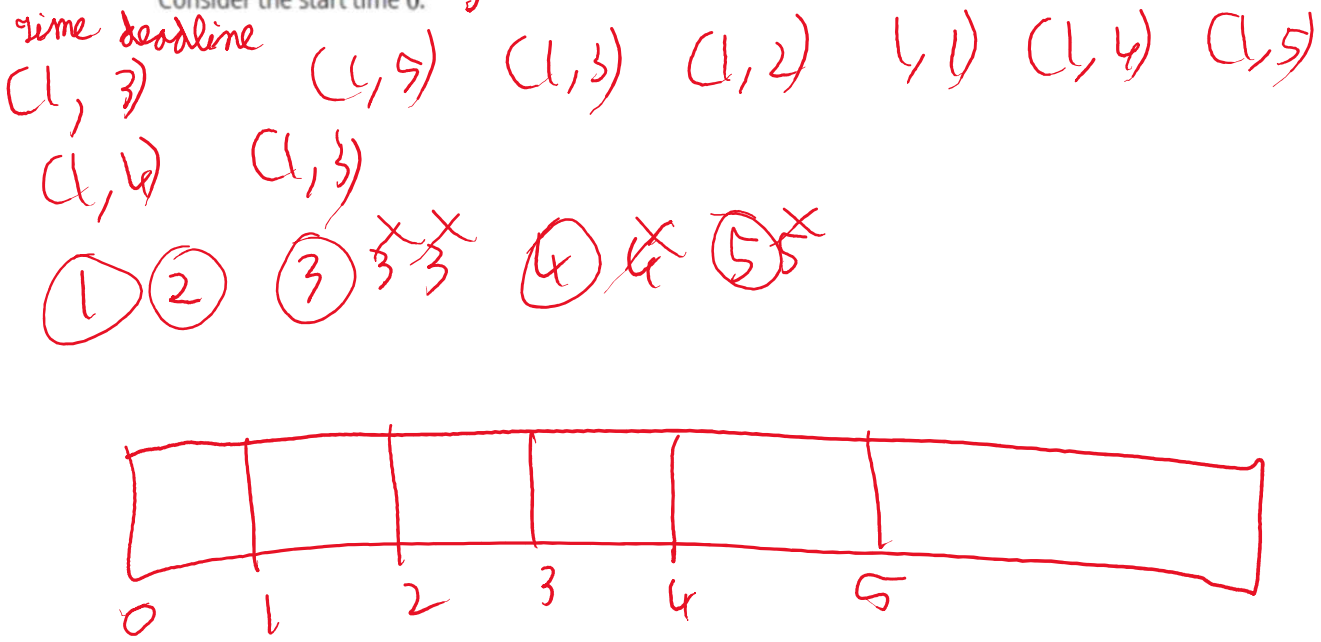


→ max heap

Consider the following tasks $T1, \ldots, T9$.

Task	T1	T2	T3	T4	T5	T6	T7	T8	T9
Deadline	3	5	3	2	1	4	5	4	3

The execution of each task requires one unit of time. We can execute one task at a time. What is the maximum number of tasks that can be completed without lateness(before or by the deadline)? Consider the start time 0.



Which of the following are possible **valid** codes for the character set S = {A, B, C, D, E, F}, generated using the Huffman algorithm?

Options :

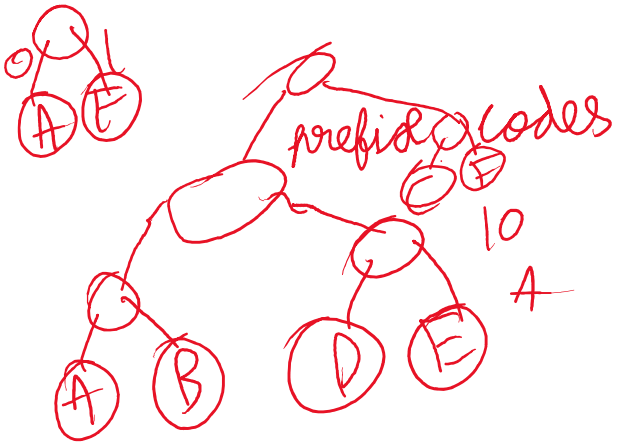
Character	A	B	C	D	E	F
Huffman code	000000	1011	110	001	01	111

Character	A	B	C	D	E	F
Huffman code	000	001	01	100	101	11

Character	A	B	C	D	E	F
Huffman code	100	1011	100	011	101	000

Character	A	B	C	D	E	F
Huffman code	0000	0001	001	01	10	11

Character	A	B	C	D	E	F
Huffman code	0000	0001	001	01	10	11



Consider the following code to find median.

```
1  def MoM(L):
2      if len(L) <= 5:
3          L.sort()
4          return L[2]
5      M = []
6      for i in range(0,len(L),5):
7          X = L[i:i+5]
8          X.sort()
9          M.append(X[2])
10     return MoM(M)
```

What median value will be returned by the given `MoM` function for the following list? 43

L = [17, 7, 76, 6, 60, 26, 12, 33, 5, 0, 49, 24, 55, 66, 75, 32, 93, 28, 43, 46, 15, 64, 50, 98, 29]

[17, 12, 55, 43, 50]

Consider n matrices $M_0, M_1, \ldots M_{n-1}$ that needs to be multiplied. Let $C[i,j]$ is the cost required to multiply matrices $M_i, M_{i+1}, \ldots, M_{j-1}, M_j$.

Let $f(x)$ is the time to multiply matrices M_x and M_{x+1}.

The optimal substructure of the problem is given as:

$$C[i,j] = \begin{cases} 0, & if\ i == j \\ \_\_\_\_, & if\ i < j \end{cases}$$

Which among the following is the correct statement to fill the blank.

Options :

[redacted] $\min_{i \le k < j} \{C[i, i+k] + C[i+k+1, j] + f(i+k)\}$

[redacted] $\min_{i \le k < j} \{C[i,k] + C[k+1,j] + f(k)\}$

[redacted] $\min_{i \le k < j} \{C[i,j] + C[k+1,j] + f(j)\}$

[redacted] None of these

The **Longest Increasing Subsequence** problem is defined as below.

Given a list `L` of size `n` non-negative integers, determine the Longest Increasing Subsequence(LIS) i.e., the longest possible subsequence in which the elements of the subsequence are sorted in increasing order.

Consider the following function `LIS` which takes list `L` as input and returns the length of the Longest Increasing Subsequence.

```
1  def LIS(L):
2      n = len(L)
3
4      Lis = [1]*n  #initialize with all 1's
5
6      for i in range(1, n):
7          for j in range(0, i):
8              if L[i] > L[j]:
9                  Lis[i] = ____ # Check here
10
11     return max(Lis)
```

Based on the above data, answer the given subquestions.

In the given code, what expression should be placed at the place of `____` so that it return the correct output?

Options :

- [redacted] `max(Lis[i], Lis[j]+1)`
- [redacted] `max(Lis[i], Lis[j])`
- [redacted] `max(Lis[i], Lis[j+1]+1)`
- [redacted] `max(Lis[i], Lis[j-1]+1)`

What is the time complexity of function `LIS()` ?

Options :

- [redacted] $O(n)$
- [redacted] $O(n \log n)$
- [redacted] $O(\log n)$
- [redacted] $O(n^2)$

so, the time complexity of this LIS function is $O(n^2)$ as it is a dp approach of finding the max element. That is why it is $O(n^2)$.

A manufacturing company produces two types of products: `A` and `B`. Market tests and available resources indicate that the combined production level should not exceed 1000 products per week and the demand for the product `B` is at most half of that for product `A`. Further, the production level of product `A` can exceed three times the production of product `B` by at most 500 units. The company makes profit of Rs 15 and Rs 20 per product respectively on products `A` and B`.

The above problem is to be formulated as a linear programming problem. Let x and y be the number of product `A` and product `B`, respectively. Objective function to maximize the number of products `z = 15x + 20y`.

Which of the following are **valid** constraints for the given problem?

Options :

[redacted] x + y <= 1000

[redacted] x - 2y >= 1

[redacted] 3x - y <= 500

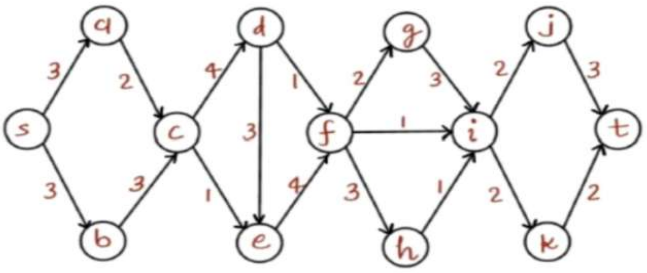
[redacted]

2x - y >=0

[redacted] x - 3y <= 500

[redacted] x, y <= 0

Consider the network given below with source *s* and sink *t*, with the numbers on the edges denoting maximum capacity across a particular edge.



Options :

6406533043496. ✖ Edges {ce, cd}

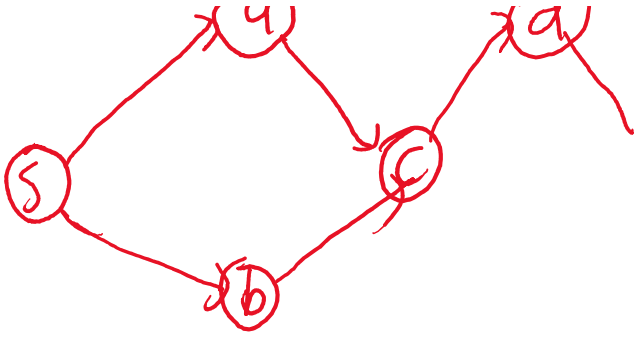
6406533043497. ✖ Edges {ac, ce, de}

6406533043498. ✔ Edges {fg, fi, hi}

6406533043499. ✖ Edges {fg, fh}

6406533043500. ✔ Edges {ij, ik}

Which of the following edges form a **valid min cut** in the given network?



Let Z be an NP-complete problem and X and Y be two other problems not known to be in NP. X is polynomial time reducible to Z and Z is polynomial-time reducible to Y. Which one of the following statements is true?

Options :

6406533043501. ✖ Y is NP-complete

6406533043502. ✔ Y is NP-hard

6406533043503. ✖ X is NP-complete

6406533043504. ✖ X is NP-hard